

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..11

SETUP

 Copy as Markdown

Set up Apple sign-in

Prerequisites

Before you start, make sure you have:

- A Serverpod project with the new auth module installed. New projects created with `serverpod create` include it by default. If you are upgrading an older project, follow the [auth module setup guide](#) first.
- An active subscription to the [Apple Developer Program](#). Sign in with Apple requires this even for local development.
- Xcode installed if you target iOS or macOS.

Get your credentials

All platforms require an App ID, a Sign in with Apple key, and a Service ID. Only the Android and web flow uses the Service ID at runtime, but sign-in initialization requires it everywhere.

Register your App ID

1. In [Certificates, Identifiers & Profiles](#), click **Identifiers** → +.
2. Select **App IDs** and click **Continue**.



Certificates, Identifiers & Profiles

[< All Identifiers](#)

Register a new identifier

Continue

- ☒ **App IDs**
Register an App ID to enable your app, app extensions, or App Clip to access available services and identify your app in a provisioning profile. You can enable app services when you create an App ID or modify these settings later.
- ☐ **Services IDs**
For each website that uses Sign in with Apple, register a services identifier (Services ID), configure your domain and return URL, and create an associated private key.

3. Select **App** as the type and click **Continue**.

4. Fill in a description and your app's **Bundle ID** (for example, `com.example.app`).

5. Scroll down to **Capabilities**, find **Sign in with Apple**, and check it. Keep it set as a **primary App ID**.

[< All Identifiers](#)

Register an App ID

Back

Continue



Sign in with Apple ⓘ

Edit

Enable as a primary App ID

6. Click **Continue**, then **Register**.

Create a Service ID

1. In Certificates, Identifiers & Profiles, click **Identifiers** → **+**.

2. Select **Services IDs** and click **Continue**.

Certificates, Identifiers & Profiles

[< All Identifiers](#)

Register a new identifier

Continue

- ☐ **App IDs**
Register an App ID to enable your app, app extensions, or App Clip to access available services and identify your app in a provisioning profile. You can enable app services when you create an App ID or modify these settings later.
- ☒ **Services IDs**
For each website that uses Sign in with Apple, register a services identifier (Services ID), configure your domain and return URL, and create an associated private key.

3. Enter a description and a unique **Identifier** (for example, `com.example.service`). This value becomes your `serviceIdentifier`. Click **Continue**, then **Register**.

4. Click on the Service ID you created above. Check **Sign in with Apple** and click **Configure**.

5. In the modal, set:

- **Primary App ID:** the App ID from the previous section
- **Domains and Subdomains:** your domain (for example, `example.com`)
- **Return URLs:** your server's callback route (for example, `https://example.com/auth/callback`)

Web Authentication Configuration

Use Sign in with Apple to let your users sign in to your app's accompanying website with their Apple Account. To configure web authentication, group your website with the existing primary App ID that's enabled for Sign in with Apple.

Primary App ID 1 App ID

Select... ▼

Register Website URLs

Provide your web domain and return URLs that will support Sign in with Apple. Your website must support TLS 1.2 or higher. All Return URLs must be registered with the https:// protocol included in the URI string. After registering new website URLs, confirm the list you'd like to add to this Services ID and click Done. To complete the process, click Continue, then click Save.

Domains and Subdomains

Enter a comma delimited list of domains and subdomains.

Return URLs

Enter a comma delimited list of Return URLs.

[Cancel](#) [Next](#)

6. Click **Next**, then **Done**, then **Save**.

⚠ WARNING

All return URLs must use **HTTPS**. Apple rejects HTTP URLs, including `localhost`. For local development, expose your server over HTTPS using a tunnelling service, like ngrok or Cloudflare Tunnel.

📖 NOTE

Register the value you will use for `appleRedirectUri`, the server callback route, under **Return URLs**. Apple validates that redirect. The separate `appleWebRedirectUri` is a page in your own web app that the server sends the browser to afterwards, so Apple never sees it and it does not belong here.

Create a Sign in with Apple key

1. In Certificates, Identifiers & Profiles, click **Keys** → **+**.
2. Enter a key name, check **Sign in with Apple**, and click **Configure**. Select your primary App ID and click **Save**.

Certificates, Identifiers & Profiles

< All Keys

Configure key Continue

Key Name

Example App Apple Sign in Key

You cannot use special characters such as @, &, *, ' ; " , - , .

Key Usage Description (optional)

ENABLE	NAME	DESCRIPTION	
☐	Apple Push Notifications service (APNs)	Establish connectivity between your notification server and the Apple Push Notification service. One key is used for all of your apps. Learn more	Edit
☐	DeviceCheck	Access the DeviceCheck and AppAttest APIs to get data that your associated server can use in its business logic to protect your business while maintaining user privacy. Learn more	
☐	Maps	Integrate Apple Maps into your apps and websites. Show maps, surface search results, provide directions, access Maps data and more. Learn more ⓘ There are no identifiers available that can be associated with the key	Configure
☐	Media Services (MusicKit, ShazamKit, Apple Music Feed)	Access the Apple Music catalog and make personalized requests for authorized users, check audio signatures against the Shazam music catalog, and access Apple Music Feed. ⓘ There are no identifiers available that can be associated with the key	Configure
☒	Sign in with Apple	Enable your apps to allow users to authenticate in your application with their Apple ID. Configuration is required to enable this feature.	Edit
☐	iWork Document Exporting	Enable your users to display iWork documents by requesting a read-only PDF version of the document. ⓘ There are no identifiers available that can be associated with the key	Configure

3. Click **Continue**, then **Register**.

Certificates, Identifiers & Profiles

ⓘ **Finish Setting up Sign in with Apple**
Depending on your product, you may need to configure multiple components for Sign in with Apple – From registering domains for Web Authentication to providing email sources to communicate with your users through the Private Email Relay service. [Learn more >](#)
[ⓘ Enable App ID](#) [ⓘ Create Service ID for Web Authentication](#) [ⓘ Create Key](#) [ⓘ Register Email Sources for Communication](#)

< All Keys

Register a New Key Back Register

Key Name

Example App Apple Sign in Key

Key Usage Description (optional)

ENABLE	NAME	DESCRIPTION
☒	Sign in with Apple	Enable your apps to allow users to authenticate in your application with their Apple ID. Configuration is required to enable this feature.

4. Download the **.p8** key file immediately: **you can only download it once**. Note the **Key ID** shown on this page.

Certificates, Identifiers & Profiles

[< All Keys](#)

Download Your Key

Download

Done



After downloading your key, it cannot be re-downloaded as the server copy is removed. If you are not prepared to download your key at this time, click Done and download it at a later time. Be sure to save a backup of your key in a secure place.

Name: Example App Apple Sign in Key

Key ID: 4RP3V2FA2L

Services: Sign in with Apple

Sign in with Apple was enabled for this key. If you haven't done so already, visit Services to complete configuration of this feature for your team.

5. Find your **Team ID** in your [Apple Developer Account](#) under Membership.

NOTE

Each primary App ID can have a maximum of two private keys. If you reach the limit, revoke an existing key before creating a new one. See [Create a Sign in with Apple private key](#) for details.

Store your credentials

Your server's `config/passwords.yaml` already has `development:`, `staging:`, and `production:` sections from the project template. Add the Apple credentials to the `development:` section:

```
development:
  # ... existing keys (database, redis, serviceSecret, etc.) ...
  appleServiceIdentifier: 'com.example.service'
  appleBundleIdentifier: 'com.example.app'
  appleRedirectUri: 'https://example.com/auth/callback'
  appleTeamId: 'ABC123DEF4'
  appleKeyId: 'XYZ789ABC0'
  appleKey: |
    -----BEGIN PRIVATE KEY-----
    MIGTAgEAMBMGBYqGSM49AgEGCCqGSM49AwEHBHkwdwIBAQQg...
    -----END PRIVATE KEY-----
  # Web only (the web app URL the server redirects the browser back to).
  appleWebRedirectUri: 'https://example.com/auth/apple-complete'
  # Android only.
  appleAndroidPackageIdentifier: 'com.example.app'
```

TIP

Paste the raw `.p8` file contents as-is. Do not pre-generate a JWT. Serverpod handles that internally. If sign-in fails, see the [troubleshooting guide](#).

When you are ready to ship, see [Publishing to production](#) for the production credential setup.

Server-side configuration

Add the Apple identity provider

⚠ CAUTION

If your project doesn't have the auth module installed yet (for example, you're upgrading an older project), follow the [auth module Setup](#) guide first before continuing.

In your server's `server.dart`, import the Apple provider and add it to the existing `identityProviderBuilders` list on `pod.initializeAuthServices()`:

```
import 'package:serverpod_auth_idp_server/providers/apple.dart';

// ...

pod.initializeAuthServices(
  tokenManagerBuilders: [
    JwtConfigFromPasswords(),
  ],
  identityProviderBuilders: [
    // ... any existing providers (e.g., EmailIdpConfigFromPasswords) ...
    AppleIdpConfigFromPasswords(),
  ],
);
```

The `AppleIdpConfigFromPasswords()` constructor reads the eight `apple*` keys from `config/passwords.yaml` (or the corresponding `SERVERPOD_PASSWORD_` environment variables), so you do not have to wire up each credential manually.

💡 TIP

If you need more control over how the credentials are loaded, you can use `AppleIdpConfig(...)` with manual `pod.getPassword()` calls instead. See the [customizations](#) page for details.

Configure web routes

Sign in with Apple requires web routes for handling callbacks and revocation notifications. Add this call before `pod.start()`:

```
pod.configureAppleIdpRoutes(
  revokedNotificationRoutePath: '/hooks/apple-notification',
  webAuthenticationCallbackRoutePath: '/auth/callback',
);
```

The `webAuthenticationCallbackRoutePath` must match the **Return URL** you registered on your Service ID. The `revokedNotificationRoutePath` is called by Apple when a user revokes access from their Apple ID settings.

Create the endpoint

Create a new endpoint file in your server project (for example, `my_project_server/lib/src/auth/apple_idp_endpoint.dart`). Extending the base class registers the sign-in methods with your server so your app can call them:

```
import 'package:serverpod_auth_idp_server/providers/apple.dart';

class AppleIdpEndpoint extends AppleIdpBaseEndpoint {}
```

Start the server

Start the server from your server project directory (for example, `my_project_server/`):

```
serverpod start
```

Then create and apply the migration for the provider's tables: in the `serverpod start` terminal, press **M** to create and apply the migration.

NOTE

Skipping the migration will cause the server to crash at runtime when the Apple provider tries to read or write user data. More detailed instructions can be found in the general [identity providers setup section](#).

Client-side configuration

The Flutter app created with `serverpod create` already has `serverpod_auth_idp_flutter` in `pubspec.yaml`. If your app doesn't, add it:

```
flutter pub add serverpod_auth_idp_flutter
```

It uses `sign_in_with_apple` under the hood for platform-specific sign-in flows.

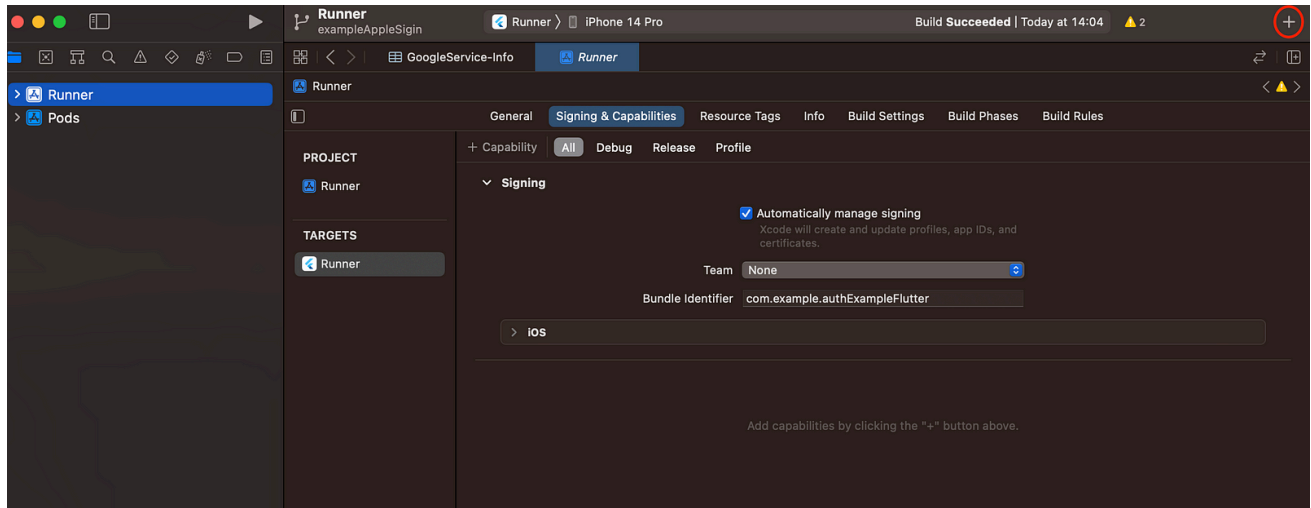
NOTE

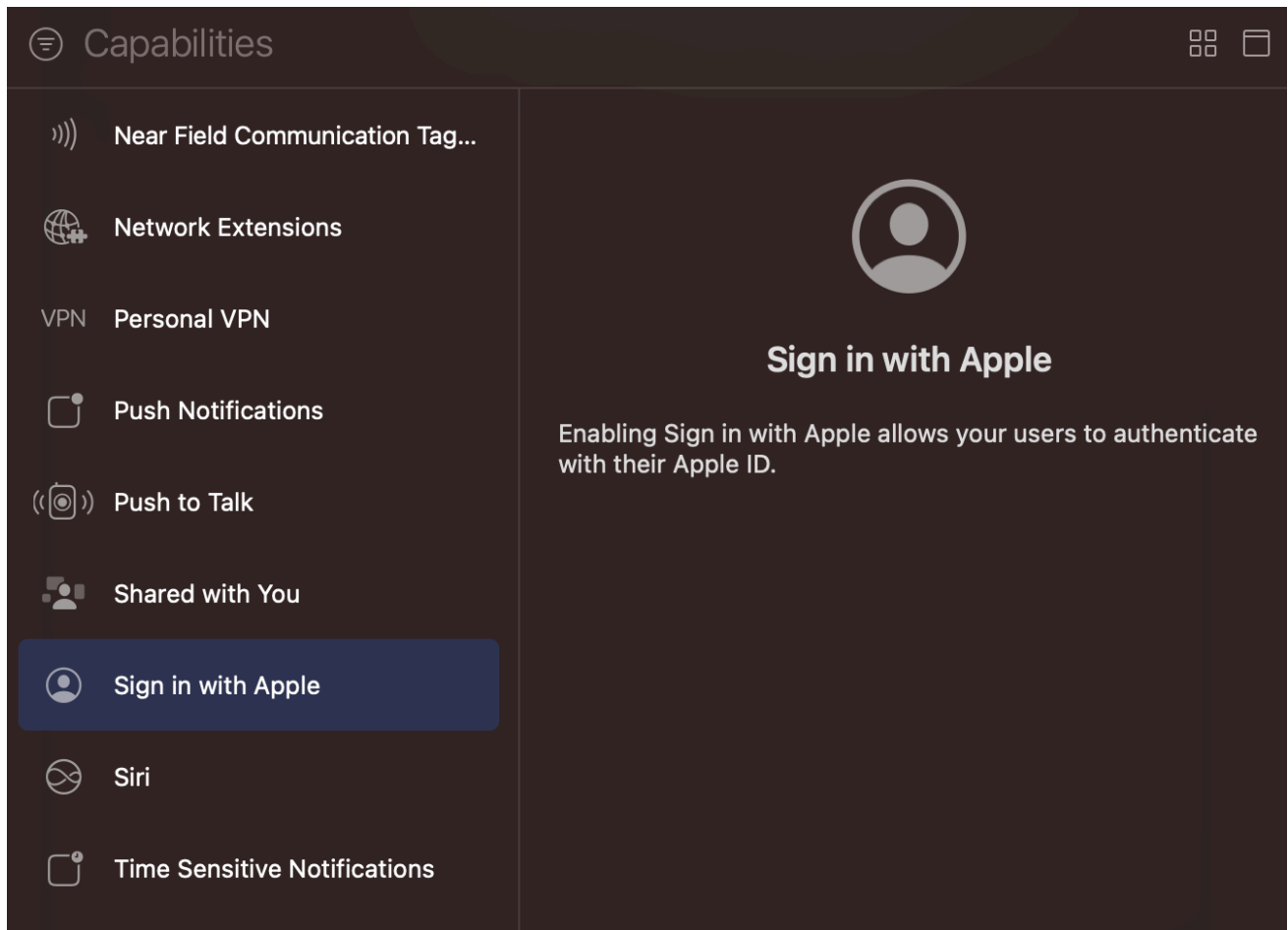
Sign in with Apple may not work correctly on all Simulator versions. If you run into issues during development, test on a physical device to confirm whether the problem is Simulator-specific.

iOS and macOS

Enable the Sign in with Apple capability in your Xcode project:

1. Open your project in Xcode.
2. Select your target.
3. Go to **Signing & Capabilities**.
4. Click + **Capability**.
5. Add **Sign in with Apple**.





Android

Sign in with Apple on Android works through a web-based OAuth flow. When the user completes authentication, Apple redirects to your server's callback route, which then redirects back to your app using an Android intent URI with the `signinwithapple` scheme.

The redirect URI and `appleAndroidPackageIdentifier` were already configured in the [Store your credentials](#) and [Service ID](#) steps. The only remaining step is to register the `signinwithapple` URI scheme in your `AndroidManifest.xml`:

```
<activity
    android:name="com.aboutyou.dart_packages.sign_in_with_apple.SignInWithAppleCallback"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="signinwithapple" />
        <data android:path="callback" />
    </intent-filter>
</activity>
```

⚠ WARNING

This intent filter is required. Without it, the OAuth callback never returns to your app and sign-in silently hangs.

Web

Sign in with Apple on web requires the Apple JS SDK. Add the following script to your Flutter app's `web/index.html` inside the `<head>` tag:

```
<script type="text/javascript" src="https://appleid.cdn-apple.com/appleauth/static/jsapi/appleid/1/en_US/appleid.auth.js"
crossorigin="anonymous"></script>
```

The redirect URI and `appleWebRedirectUri` were already configured in the [Store your credentials](#) and [Service ID](#) steps.

Present the authentication UI

Initialize the Sign in with Apple service

In your Flutter app's `lib/client.dart`, the template already sets up the `Client` and calls `client.auth.initialize()` inside `initializeClient()`. Add `client.auth.initializeAppleSignIn()` right after it:

```
unawaited(client.auth.initialize());
client.auth.initializeAppleSignIn();
```

The sign-in service needs your Service ID and redirect URI on every platform, even though only the web and Android flow uses them. Pass them as build-time environment variables using `--dart-define`:

```
flutter run \
  -d "<device>" \
  --dart-define="APPLE_SERVICE_IDENTIFIER=com.example.service" \
  --dart-define="APPLE_REDIRECT_URI=https://example.com/auth/callback"
```

Use the same values you configured in the [Service ID](#) and [Store your credentials](#) steps.

You can also pass the values directly as parameters instead. See the [customizations page](#) for details.

Add the sign-in widget

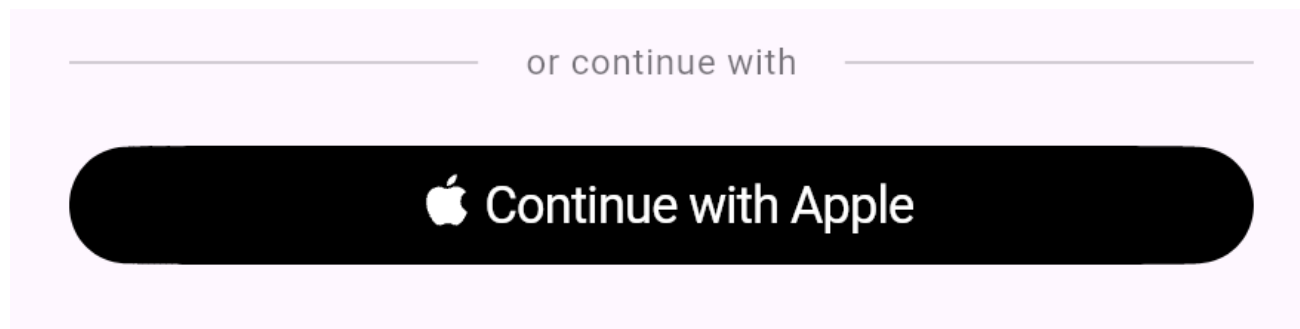
If you have configured the `SignInWidget` as described in the [setup section](#), the Apple identity provider will be automatically detected and displayed in the sign-in widget.

You can also use the `AppleSignInWidget` directly in your widget tree to include the Apple authentication flow in your own custom UI:

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

AppleSignInWidget(
  client: client,
  onAuthenticated: () {
    // Do something when the user is authenticated.
    //
    // NOTE: You should not navigate to the home screen here, otherwise
    // the user will have to sign in again every time they open the app.
  },
  onError: (error) {
    // Handle errors
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Error: $error')),
    );
  },
)
```

This renders a Sign in with Apple button like this:



The widget automatically handles:

- Sign in with Apple flow for iOS, macOS, Android, and web.
- Token management.
- Underlying `sign_in_with_apple` package error handling.

For details on how to customize the Sign in with Apple UI in your Flutter app, see the [customizations page](#).

⚠ WARNING

Apple sends the user's email and name only on the **first sign-in**. Serverpod stores them automatically during that first authentication. If the first sign-in never reaches your server, they cannot be retrieved

later. See [User email is null after sign-in](#).

Publishing to production

Update the Apple Developer Portal

Add your production domain and callback URL to the Service ID. The development tunnel URL and the production URL can stay registered at the same time.

1. In [Certificates, Identifiers & Profiles](#), open your Service ID.
2. Under **Sign in with Apple**, click **Configure**.
3. Add your production domain to **Domains and Subdomains** (for example, `example.com`).
4. Add your production callback to **Return URLs** (for example, `https://example.com/auth/callback`).
5. Click **Next**, **Done**, then **Save**.

Set production credentials

Production runs out of the `production:` section of `passwords.yaml`, which is separate from the `development:` section you populated during setup. Adding production credentials does not replace your development ones. Both stay in place, and Serverpod picks the right set based on the run mode.

Most credentials, like the Team ID, Key ID, and `.p8` private key, can be reused from development. The values that typically differ are the URLs (`appleRedirectUri` and `appleWebRedirectUri`), which should point at your production domain rather than your development tunnel. If you use a different App ID or Service ID for production, register them in the [Apple Developer Portal](#) first and use those identifiers below.

Pick the path that matches your deployment:

Self-hosted

Add the `apple*` keys to the `production:` section of `passwords.yaml` with production values, or set them as environment variables on the production server using the `SERVERPOD_PASSWORD_` prefix (for example, `SERVERPOD_PASSWORD_appleServiceIdentifier`, `SERVERPOD_PASSWORD_appleKey`).

Serverpod Cloud

Use `serverpod cloud password set` for each credential. The `appleKey` value spans multiple lines, so pass it via `--from-file`:

```
serverpod cloud password set appleServiceIdentifier "com.example.service"
serverpod cloud password set appleBundleIdentifier "com.example.app"
```

```
serverpod cloud password set appleRedirectUri "https://example.com/auth/callback"
serverpod cloud password set appleTeamId "ABC123DEF4"
serverpod cloud password set appleKeyId "XYZ789ABC0"
serverpod cloud password set appleKey --from-file ./AuthKey_XYZ789ABC0.p8
serverpod cloud password set appleWebRedirectUri "https://example.com/auth/apple-
complete"
serverpod cloud password set appleAndroidPackageIdentifier "com.example.app"
```

Run these from your linked server project directory, or pass `--project <project-id>` on each call. See the [Serverpod Cloud passwords guide](#) for project linking and other options.

Update app builds

Release builds need the production Service ID and redirect URI on every platform, the same as during development. Pass them via `--dart-define`:

```
flutter build web \
  --dart-define="APPLE_SERVICE_IDENTIFIER=com.example.service" \
  --dart-define="APPLE_REDIRECT_URI=https://example.com/auth/callback"
```

TIP

If you run into issues, see the [troubleshooting guide](#).